Assignment 9: Random Numbers

PHYS 4202 - Computational Physics (Fall 2026)
Due: Tuesday, October 27, 2026 at 10:00 am
Total points: 100 (+20 optional extra credit)

Purpose

Use pseudo-random numbers to model individual events and investigate the patterns that emerge from many events. Simulate the Monty Hall problem, follow radioactive nuclei as they decay, and compare aggregate results with probability predictions. The emphasis is on:

- translating a problem's rules into correct random choices
- distinguishing one random outcome from an aggregate probability or average
- interpreting running probability estimates, decay histories, and half-lives
- checking simulations against known results and limiting cases
- making random experiments reproducible

Environment

All work must be done in **Python 3.9 or newer**. You may use the Python standard library, NumPy, and matplotlib. Use `numpy.random.default_rng` and allow a user-specified seed. Create a generator once per script run and pass it to simulation functions. Do not reset the seed inside a trial or timestep loop. Record the seeds and parameters used for the reported results.

Code organization

Your code directory must contain the following Python files:

- `monty_hall.py`: runs the Part 1 Monty Hall simulations, analyzes the results, and generates its plots.
- `decay.py`: runs the Part 2 radioactive decay simulations, estimates half-lives, and generates its plots.
- `airplane.py`: runs the airplane seating simulations, analyzes the results, and generates its plots, required only if completing the optional extra credit.

Put reusable simulation logic in functions and allow the main parameters to be changed without editing those functions. Additional helper modules are optional. Include a brief `README.md` with run instructions, dependencies, parameters, and seed usage.

Validate inputs and raise helpful errors for invalid values, including noninteger counts, invalid population sizes, and nonpositive trial counts or timesteps. Counts must be integers. Type hints are encouraged but not required.

Part 1: The Monty Hall Problem (40 points)

Rules

There are three doors: one hides a prize and two hide goats. The contestant picks one door and the host, who knows where the prize is located, opens another door to reveal a goat. The contestant can either stay with the initial choice or switches to the other unopened door. For three doors, the predicted win probabilities are 1/3 for staying and 2/3 for switching.

A generalized version of this problem is to have $N \geq 2$ doors and have the host open $N - 2$ doors, never opening the contestant's choice or the prize door. Exactly two doors remain unopened. If the initial choice contains the prize, the host chooses randomly among the other doors which one to leave unopened. The host opens all other doors. If the initial choice does not contain the prize, the prize door is the other unopened door. For $N = 2$, the host opens no doors.

Experiments and analysis

Support a user-specified number of trials k, with default $k = 10{,}000$. For each $N \in \{2, 3, 10, 100\}$, run separate experiments for always staying and always switching. Use separate random trials and outcomes for the two strategies. Do not calculate one strategy's measured win fraction as one minus the other's.

For each strategy calculate the running win fraction after j trials,

$$f_{\text{win}}(j) = \frac{\text{number of wins in } j \text{ trials}}{j}$$

- Make a combined figure with one panel per N, showing both strategies in each panel. Use a logarithmic trial-number axis, beginning at $j = 1$.
- Derive the predicted stay and switch probabilities for general N and add them as horizontal reference lines.
- Tabulate the final measured probabilities for both strategies at every required N and compare to the predicted values of $1 - (1/N)$ for switching.
- Discuss agreement, early fluctuations, and why a running estimate need not approach its limiting value monotonically.
- Check $N = 2$ explicitly and explain the result.

Part 2: Radioactive Decay (45 points)

Model and random update

Start with $M_0 = 100{,}000$ identical radioactive nuclei. During each timestep, every surviving nucleus independently has probability p of decaying. A decayed nucleus cannot decay again.

For every surviving nucleus at each timestep, draw a fresh uniform random number $u \in [0, 1)$. If $u < p$, remove that nucleus from the surviving population. Record the number of undecayed nuclei after each update, including $M(0) = M_0$. Vectorized draws for the surviving population are allowed

and encouraged. Direct calls to a binomial sampler or an exponential lifetime sampler are not allowed for the required simulation.

Use a timestep $\Delta t = 1.0$ s and run for 200 steps, through $t_f = 200$ s. For each of the two per-step decay probabilities

$$p = 0.10 \qquad \text{and} \qquad p = 0.02$$

run **five independent simulations**, each starting with $M_0 = 100{,}000$ nuclei. Reset the population, but not the random generator, between runs. Support user-specified integer $M_0 \geq 1$, $0 \leq p < 1$, $\Delta t > 0$, and a positive integer number of steps. The required settings should be small enough for efficient simulations with NumPy random arrays. Only survivor counts need to be stored, not individual nucleus histories.

Decay probability, decay constant, and expected population

A per-step probability p is dimensionless and refers to a particular interval Δt. The corresponding decay constant λ, with units of inverse time, is

$$p = 1 - e^{-\lambda \Delta t} \qquad \lambda = -\frac{\ln(1-p)}{\Delta t}$$

After n steps, the expected number of survivors is

$$M_{\text{expected}}(t_n) = M_0(1-p)^n = M_0 e^{-\lambda t_n}$$

- Make a figure with one panel per decay probability. Plot the **number of remaining nuclei** versus time for all five runs in each panel, using a logarithmic y-axis. Include $t = 0$ and label time in seconds. Omit zero counts from the logarithmic plot.
- At each time, calculate the average population across the five runs, $\overline{M}(t) = \frac{1}{5}\sum_{r=1}^{5} M_r(t)$. Add this average as a clearly distinguished curve, together with $M_{\text{expected}}(t)$.
- Discuss agreement with the expected decay curve, differences among individual runs, and how changing p changes the decay rate. With this large initial population, curves may nearly overlap, so use sufficiently thin lines.

Estimating the half-life

For each probability, estimate the half-life from the **average of the five simulated population histories**. Find the sampled time at which $\overline{M}(t)$ is closest to $M_0/2$. Use that time as $T_{1/2}^{\text{measured}}$. If two sampled times are equally close, use the earlier time.

The predicted half-life is

$$T_{1/2}^{\text{predicted}} = \frac{\ln 2}{\lambda} = -\frac{\Delta t \ln 2}{\ln(1-p)}$$

- Mark $M_0/2$ and the measured half-life on each decay panel.
- Report p, λ, the predicted and measured half-lives, and their relative difference $(T_{1/2}^{\text{measured}} - T_{1/2}^{\text{predicted}})/T_{1/2}^{\text{predicted}}$. Include units where appropriate.

Checks

Check that survivor counts remain integers between zero and M_0 and never increase. Verify that $p = 0$ leaves the population unchanged and that an extinct population stays at zero for later steps. These checks may use separate small runs or an update function called with zero survivors. Summarize the checks in the report.

Optional Extra Credit: Airplane Seating (+20 points)

An airplane has N seats and N passengers, with passenger i assigned to seat i. Passengers board in order $1, 2, \ldots, N$.

- Passenger 1 has forgotten their seat number and chooses uniformly from all available seats.
- Each subsequent passenger sits in their assigned seat if it is available. Otherwise, they choose uniformly from the remaining available seats.

What is the probability that the last passenger sits in their assigned seat? Simulate the boarding process explicitly, resetting the available seats for each trial. Do not replace boarding with a direct draw for the final passenger's outcome.

Support integer $N \geq 2$ (default $N = 100$) and a user-specified number of independent trials k (default $k = 10{,}000$). Run the experiment for $N \in \{10, 100, 500, 1000\}$. A trial is a success if the last passenger sits in their assigned seat. Let $S(j)$ be the number of successes in the first j trials. The running probability estimate is

$$\widehat{p}(j) = \frac{S(j)}{j}$$

Plot the running probability versus trial number using a logarithmic horizontal axis. Show all four N values in a combined figure, using separate panels if helpful. Tabulate the final estimates and discuss in the report how the probability depends on N, supporting your conclusion with the simulations. Explain the behavior you observe. Include the extra credit figure and discussion in the same report as the required work.

Figures, short report, and submission (15 points)

Prepare a concise report in LaTeX and submit the compiled PDF directly to Canvas. Include your name, a descriptive title, and your GitHub repository URL. Organize the discussion as a connected analysis supported by the requested figures, tables, quantitative comparisons, and checks. Describe how random events produce the aggregate results in each part. Include a brief testing or implementation note.

Include every required figure in the report, including the extra credit figure if attempted. Label axes, identify curves, and provide concise captions. Label the decay population as a number of nuclei and time in seconds. Use logarithmic horizontal axes for the running probabilities in Part 1 and the extra credit, and linear time axes for radioactive decay. Combine panels where they remain readable.

Include an **AI-use statement**. If you used AI, name the tools, describe their contributions, and explain how you checked or revised the resulting work. If you did not use AI, say so.

Commit the Python code, supporting modules, and `README.md` to your GitHub repository. Submit the compiled report PDF to Canvas. Do not submit plot files separately or commit them to the repository. Keep locally generated figures out of version control using `.gitignore`. The figures are assessed within the report.

Grading (100 points, plus 20 optional extra credit)

Part 1: The Monty Hall problem (40 points)

Category	What I look for	Pts
Simulation logic	Correct random choices, rules, and independent strategy runs	15
Generalization and checks	Required door counts, input validation, and $N = 2$ check	10
Probability analysis	Running probability figure, theory, results, and interpretation	15
Total		40

Part 2: Radioactive decay (45 points)

Category	What I look for	Pts
Decay implementation	Per-survivor draws, relation between p and λ, and time updates	15
Population histories	Five runs per p value, average curves, and interpretation	10
Half-life analysis	Measured value and and quantitative comparison to prediction	15
Checks	Parameter validation, monotonicity, zero decay, and extinction	5
Total		45

Short report and documentation (15 points)

Category	What I look for	Pts
Presentation	Clear figures, labels, captions, and organization	5
Discussion	Analysis supported by results and checks	5
Documentation	README, repo URL, testing note, and AI statement	5
Total		15

Optional extra credit: Airplane seating (+20 points)

Category	What I look for	Pts
Simulation	Correct boarding, uniform choices, and validation checks	10
Analysis	Running probabilities, results, and discussion of N dependence	10
Total extra credit		20